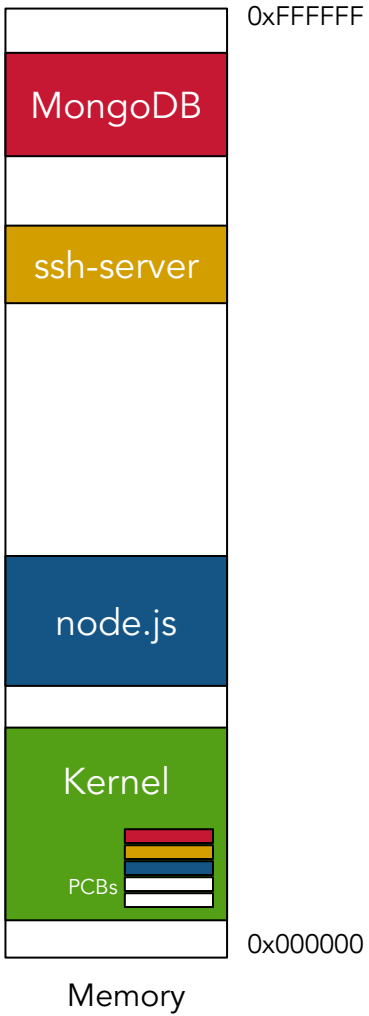
Lecture 4:

Concurrency & Threads

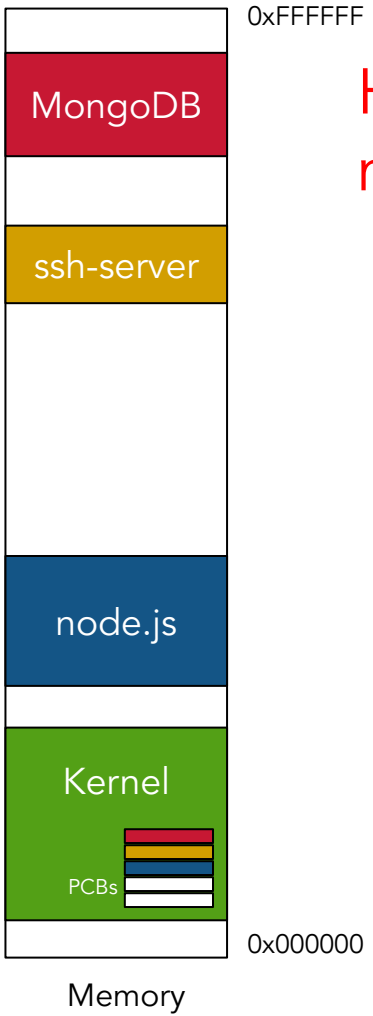
CS211 - Operating systems

Spring 2026

Example

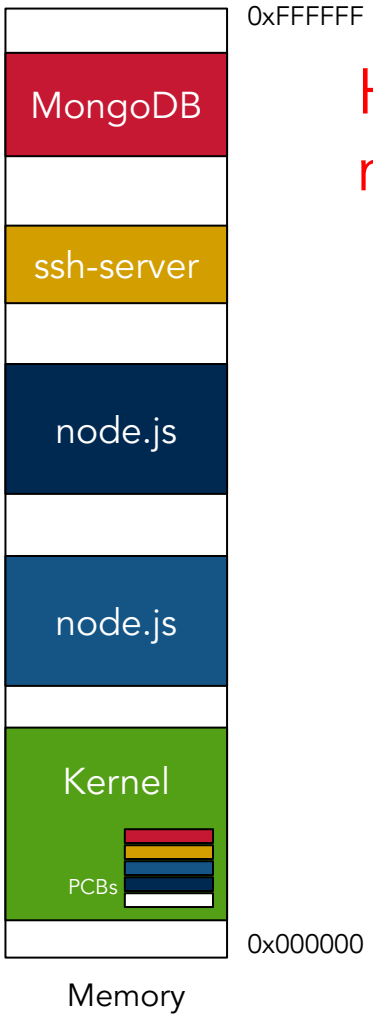

0xFFFFFF
MongoDB
ssh-server
node.js
Kernel
PCBs
0x000000
Memory

Example



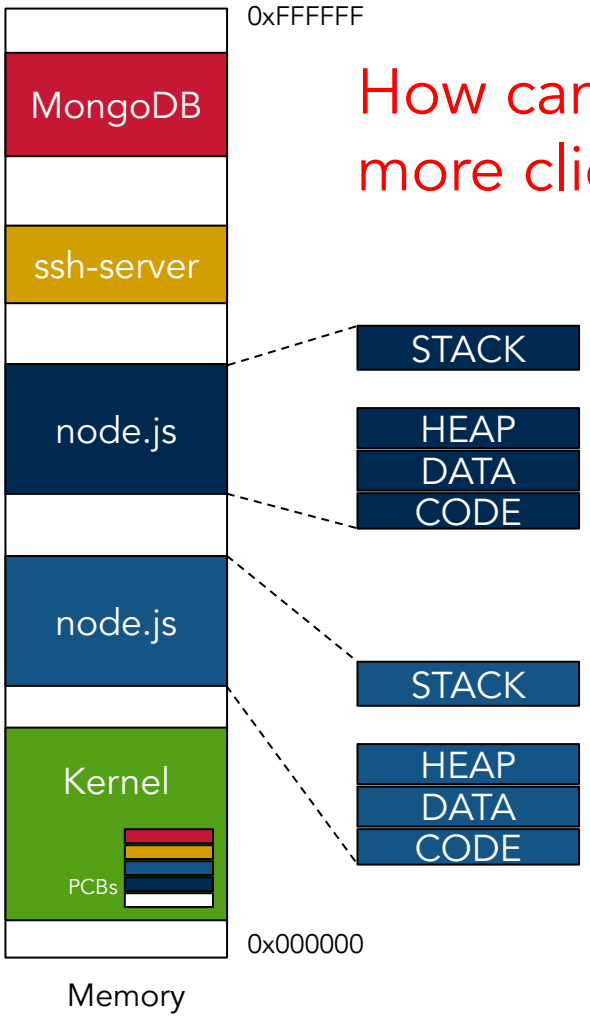
How can Node.js handle more client requests?

Example



How can Node.js handle more client requests?

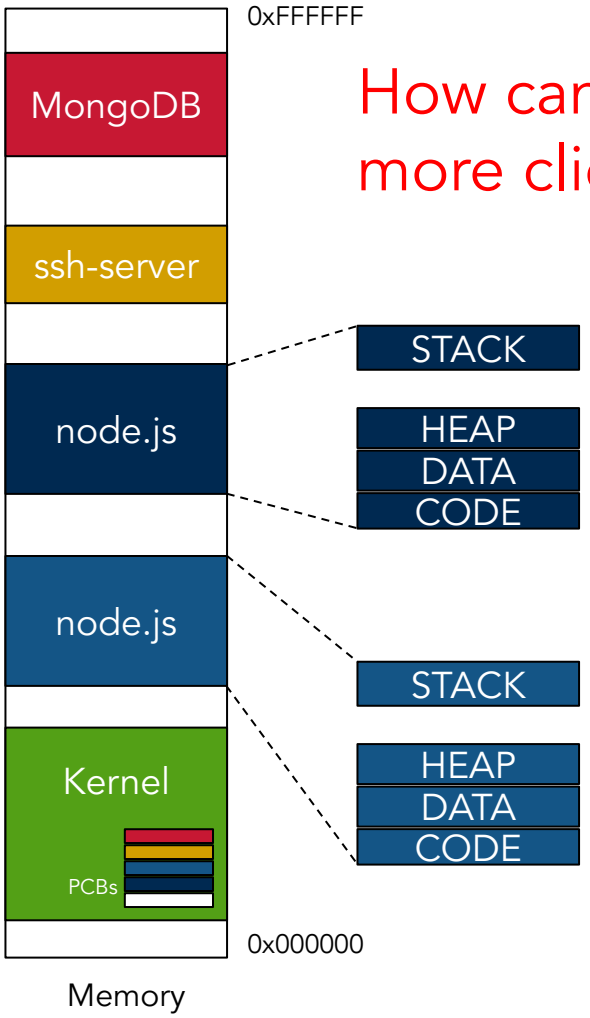
Example



How can Node.js handle more client requests?

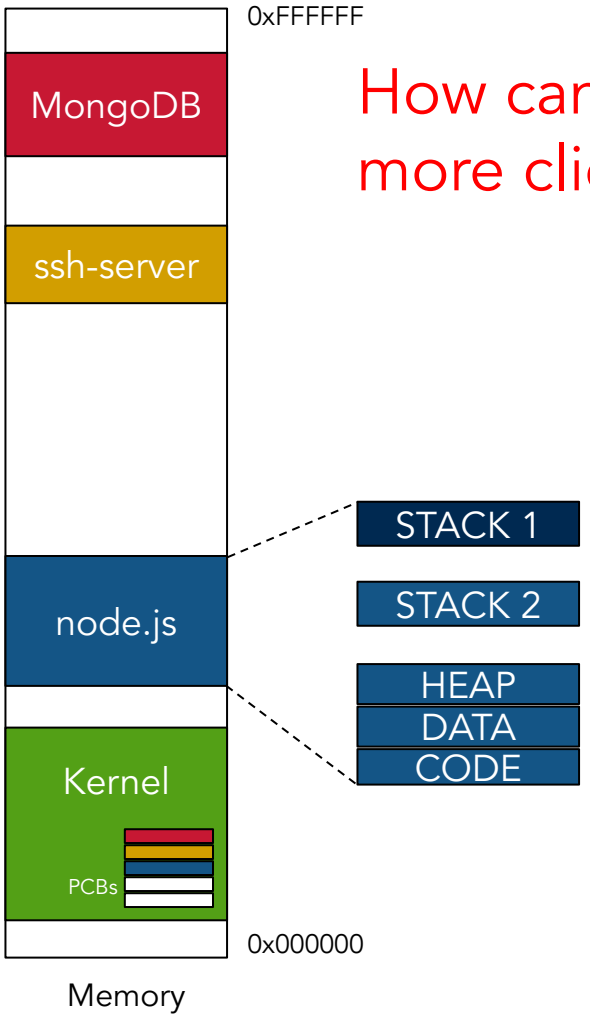
Example

1. Costs memory
2. Costs management time
3. Hard to communicate



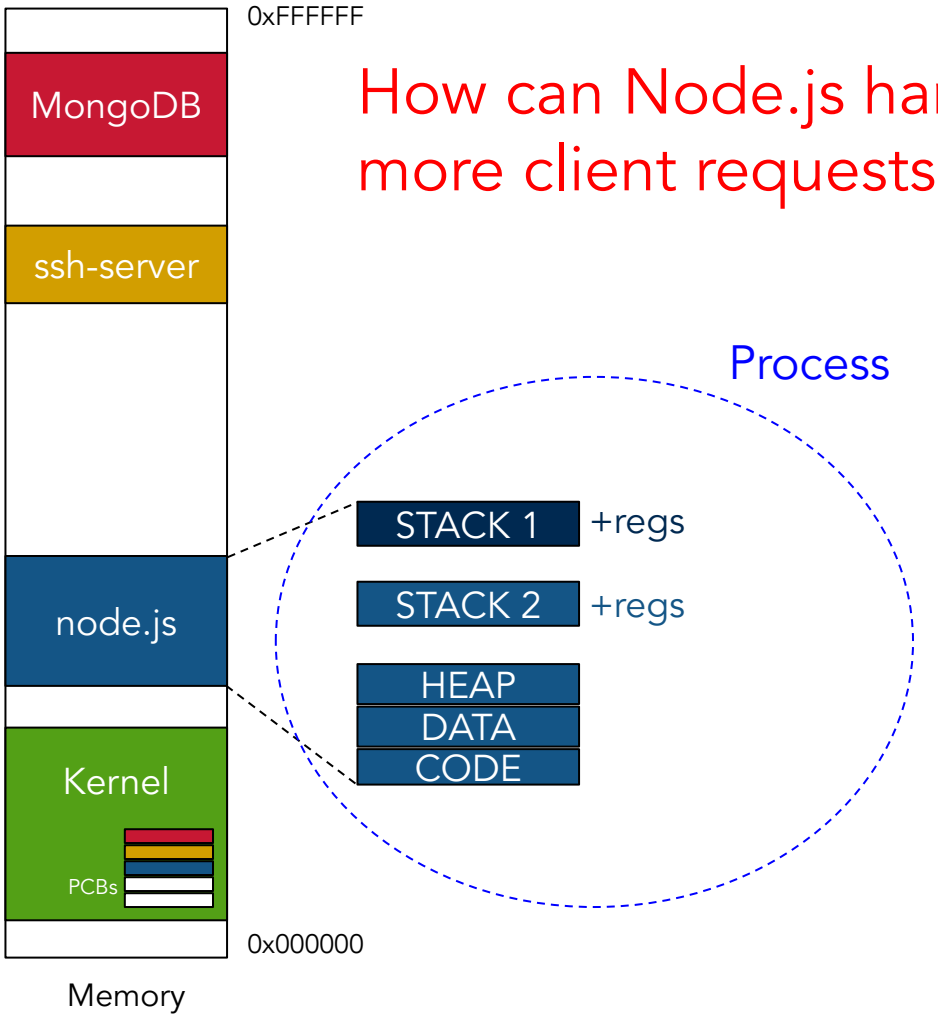
How can Node.js handle more client requests?

Example



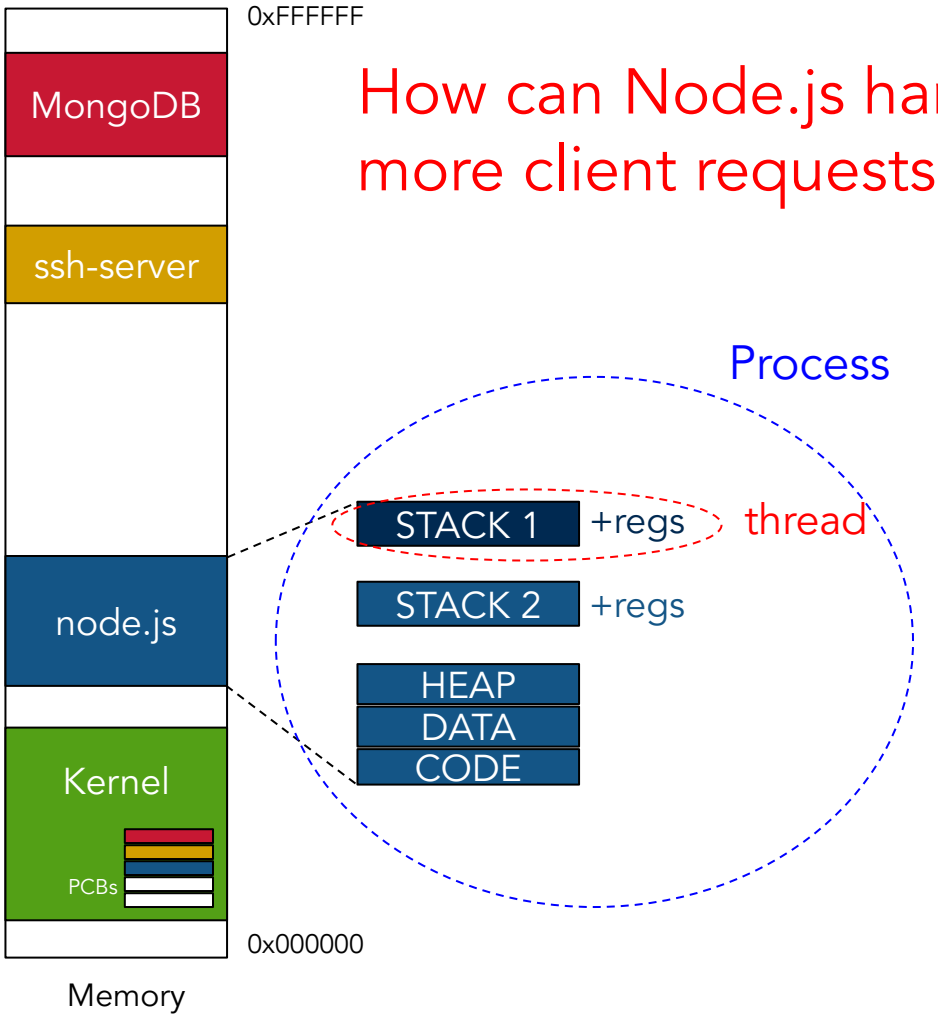
How can Node.js handle more client requests?

Example



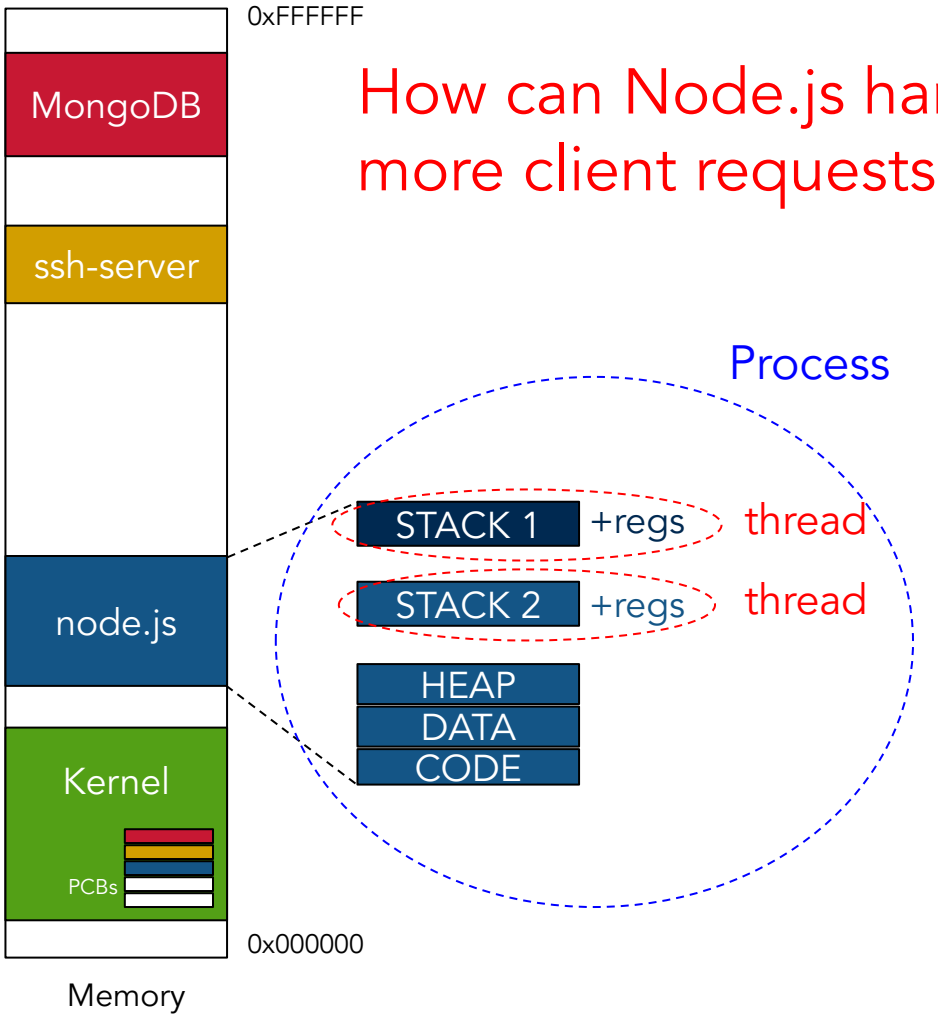
How can Node.js handle more client requests?

Example



How can Node.js handle more client requests?

Example

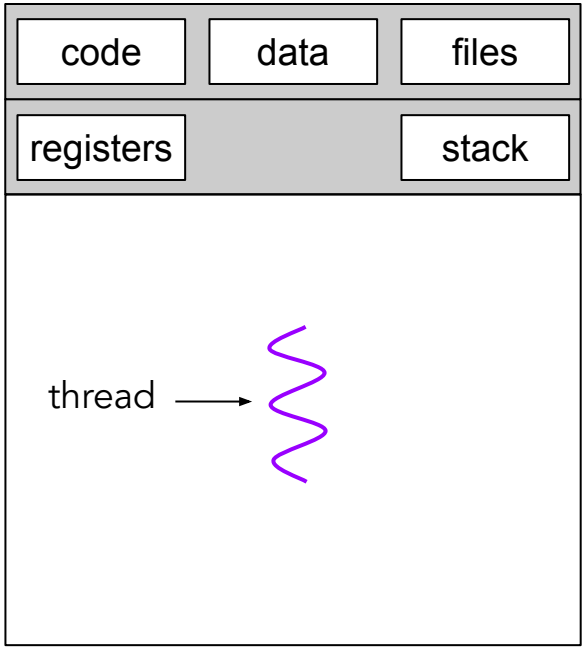


How can Node.js handle more client requests?

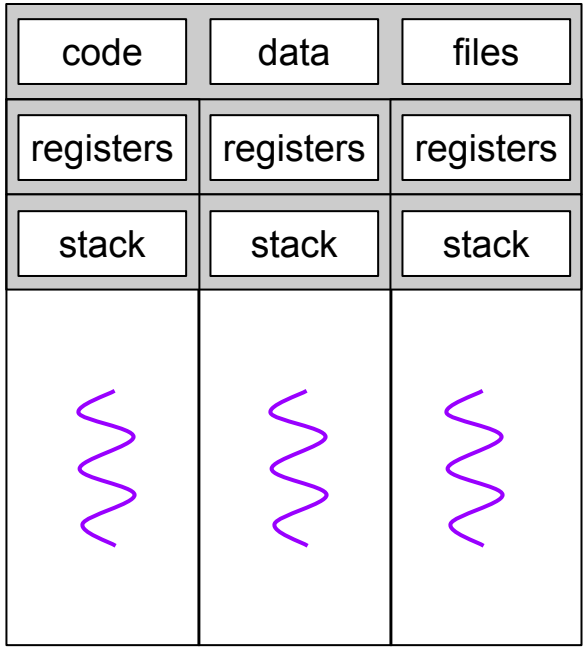
Threads

- Definition
  - Sequence of instructions on a single processor
- Processes vs threads
  - A process is an abstraction of a computer
    - CPU, memory, devices
  - A thread is an abstraction of a core
    - registers (incl. PC and SP)

Single-threaded vs multithreaded processes



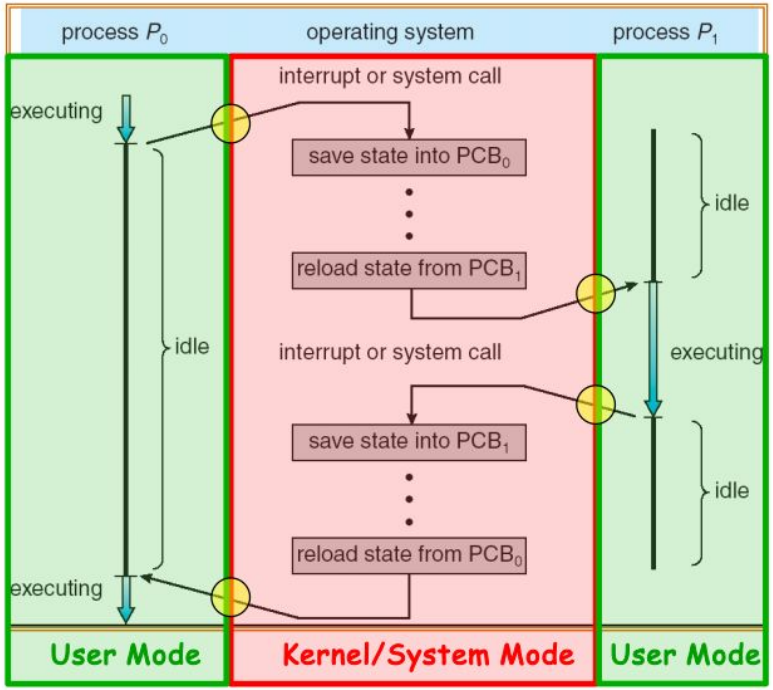
single-threaded process



multithreaded process

Why threads?

L1 cache reference	0.5 ns
Branch mispredict	5 ns
L2 cache reference	7 ns
Mutex lock/unlock	25 ns
Main memory reference	100 ns
Compress 1K bytes with Zippy	3,000 ns
Send 2K bytes over 1 Gbps network	20,000 ns
Read 1 MB sequentially from memory	250,000 ns
Round trip within same datacenter	500,000 ns
Disk seek	10,000,000 ns
Read 1 MB sequentially from disk	20,000,000 ns
Send packet CA->Netherlands->CA	150,000,000 ns



Processes

- Cost memory
- Cost management time
- Hard to communicate
- Hard to switch contexts

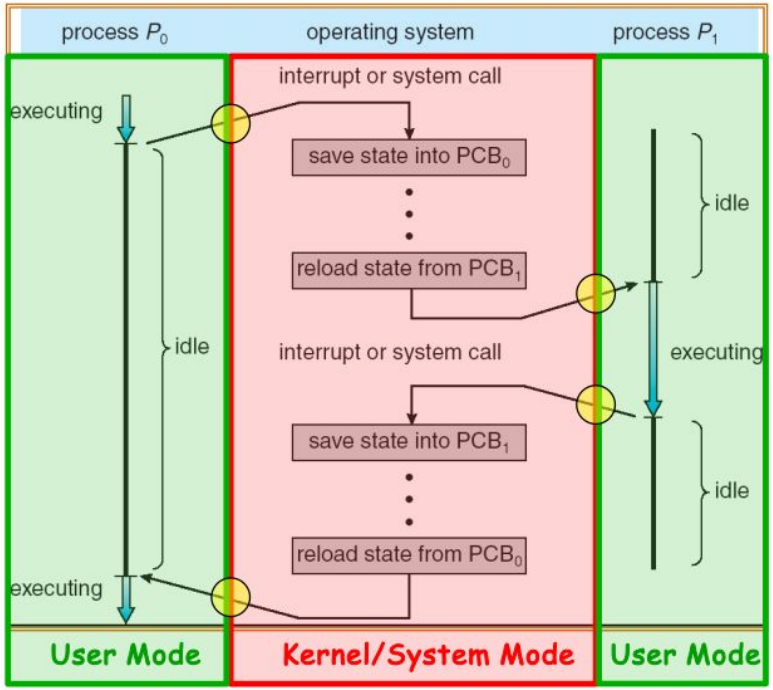
Why threads?

L1 cache reference	0.5 ns
Branch mispredict	5 ns
L2 cache reference	7 ns
Mutex lock/unlock	25 ns
Main memory reference	100 ns
Compress 1K bytes with Zippy	3,000 ns
Send 2K bytes over 1 Gbps network	20,000 ns
Read 1 MB sequentially from memory	250,000 ns
Round trip within same datacenter	500,000 ns
Disk seek	10,000,000 ns
Read 1 MB sequentially from disk	20,000,000 ns
Send packet CA->Netherlands->CA	150,000,000 ns

~~Processes~~ Threads

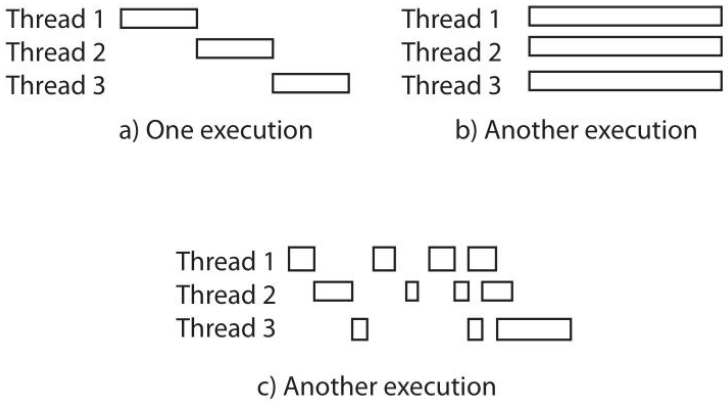
Cost less
- ~~Cost~~ memory
- ~~Cost~~ management time

Easy
- ~~Hard~~ to communicate
- ~~Hard~~ to switch contexts



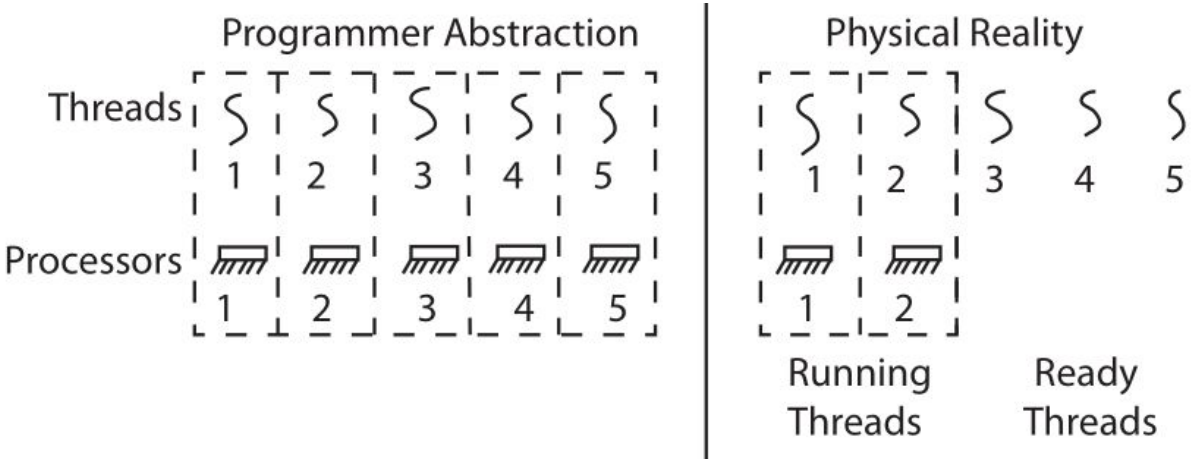
Concurrency

- Concurrency vs parallelism
  - Concurrency is about handling multiple things at once



  - Parallelism is about doing multiple things simultaneously

Illusion vs reality



Programmer vs. Processor View

Programmer's View	Possible Execution #1	Possible Execution #2	Possible Execution #3
.	.	.	.
.	.	.	.
.	.	.	.
x = x + 1;	x = x + 1;	x = x + 1	x = x + 1
y = y + x;	y = y + x;	.....	y = y + x
z = x +5y;	z = x + 5y;	thread is suspended	.....
.	.	other thread(s) run	thread is suspended
.	.	thread is resumed	other thread(s) run
.	.	.....	thread is resumed
		y = y + x	.....
		z = x + 5y	z = x + 5y

Preemption

- Two kinds of threads
  - Non-preemptive: explicitly yield to other threads
  - Preemptive: yield automatically upon clock interrupts
    - Most modern threading systems

How to create a thread?

How to create a thread?

Functions	Description
fork() + exec(prog, args)	Create a new process and run a new program
wait(&status)	pause until a child process has exited
exit(status)	Tell the kernel the current process is complete and should be garbage collected

How to create a thread?

Functions	Description
fork() + exec(prog, args) int pthread_create(pthread_t *thread, const pthread_attr_t *attr, void *(*start_routine)(void*), void *arg);	Create a new process and run a new program thread is created and executes the start_routine with arg as its sole argument, return is implicit call to pthread_exit, (attr contains info like stack size, scheduling policy, etc)
wait(&status)	pause until a child process has exited
exit(status)	Tell the kernel the current process is complete and should be garbage collected

"p" is for "POSIX" which is a part of a standardized API
https://pubs.opengroup.org/onlinepubs/7908799/xsh/pthread.h.html

How to create a thread?

Functions	Description
fork() + exec(prog, args) int pthread_create(pthread_t *thread, const pthread_attr_t *attr, void *(*start_routine)(void*), void *arg);	Create a new process and run a new program thread is created and executes the start_routine with arg as its sole argument, return is implicit call to pthread_exit, (attr contains info like stack size, scheduling policy, etc)
wait(&status) int pthread_join(pthread_t thread, void **value_ptr);	pause until a child process has exited suspends execution of the calling thread until the target thread terminates. On return with a non-NULL value_ptr the value passed to pthread_exit() by the terminating thread is made available in the location referenced by value_ptr.
exit(status)	Tell the kernel the current process is complete and should be garbage collected

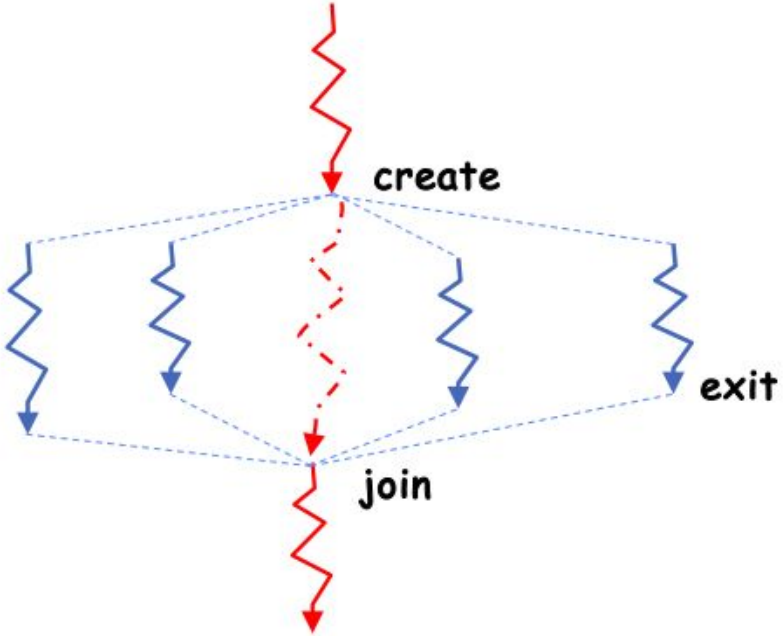
"p" is for "POSIX" which is a part of a standardized API
https://pubs.opengroup.org/onlinepubs/7908799/xsh/pthread.h.html

How to create a thread?

Functions	Description
fork() + exec(prog, args) int pthread_create(pthread_t *thread, const pthread_attr_t *attr, void *(*start_routine)(void*), void *arg);	Create a new process and run a new program thread is created and executes the start_routine with arg as its sole argument, return is implicit call to pthread_exit, (attr contains info like stack size, scheduling policy, etc)
wait(&status) int pthread_join(pthread_t thread, void **value_ptr);	pause until a child process has exited suspends execution of the calling thread until the target thread terminates. On return with a non-NULL value_ptr the value passed to pthread_exit() by the terminating thread is made available in the location referenced by value_ptr.
exit(status) void pthread_exit(void *value_ptr);	Tell the kernel the current process is complete and should be garbage collected Terminates the thread and makes value_ptr available to any successful join

"p" is for "POSIX" which is a part of a standardized API
https://pubs.opengroup.org/onlinepubs/7908799/xsh/pthread.h.html

Thread operations



Implementation of threads

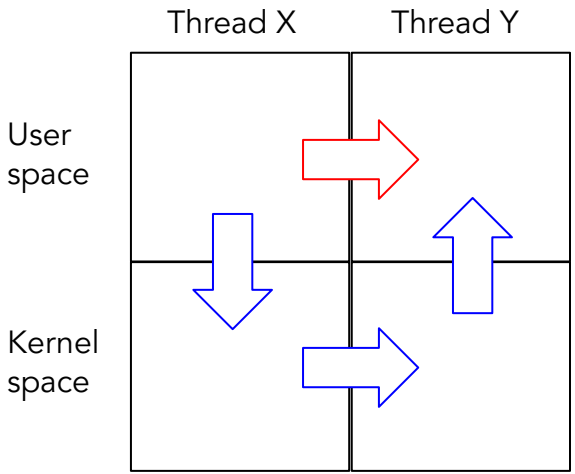
- "kernel threads"
  - each thread has its own PCB in the kernel, but the PCBs point to the same physical memory
- "user threads":
  - one PCB for the process; threads implemented entirely in user space. Each thread has its own Thread Control Block (TCB) and context

Kernel threads vs user threads

	Kernel threads	**User threads**
Implementation	Slightly changed from the process management	Need to implement context switch, scheduler, etc
Blocking system calls	Blocking system calls do not block other threads	Blocking system calls block all threads in the process
Context switch	Need 3 context switches	Need 1 context switch only

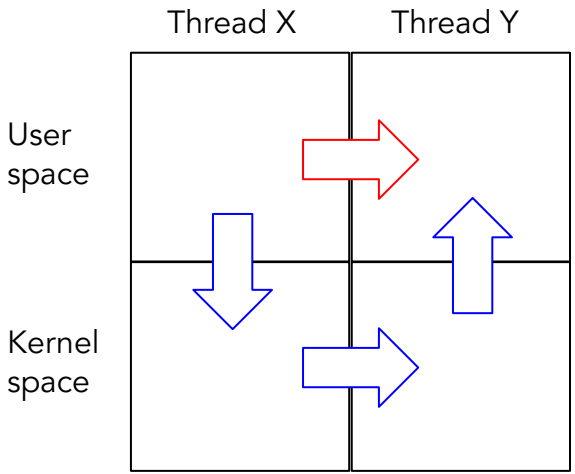
Kernel threads vs user threads

	Kernel threads	**User threads**
Implementation	Slightly changed from the process management	Need to implement context switch, scheduler, etc
Blocking system calls	Blocking system calls do not block other threads	Blocking system calls block all threads in the process
Context switch	Need 3 context switches	Need 1 context switch only



Kernel threads vs user threads

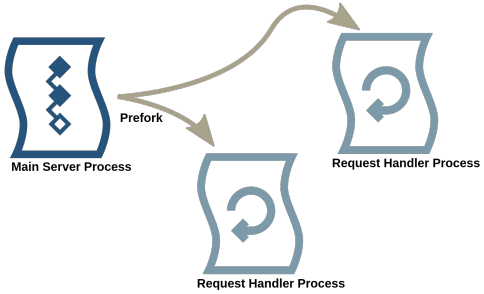
	Kernel threads	**User threads**
Implementation	Slightly changed from the process management	Need to implement context switch, scheduler, etc
Blocking system calls	Blocking system calls do not block other threads	Blocking system calls block all threads in the process
Context switch	Need 3 context switches	Need 1 context switch only



Thread-based vs event-based programming

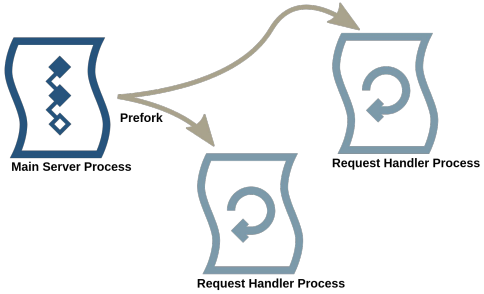
https://berb.github.io/diploma-thesis/community/042_serverarch.html

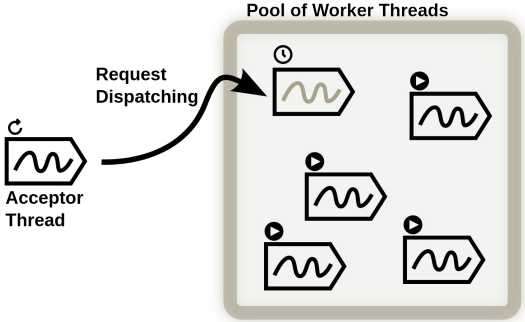
Thread-based vs event-based programming



https://berb.github.io/diploma-thesis/community/042_serverarch.html

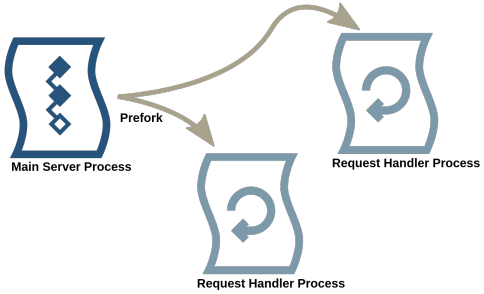
Thread-based vs event-based programming

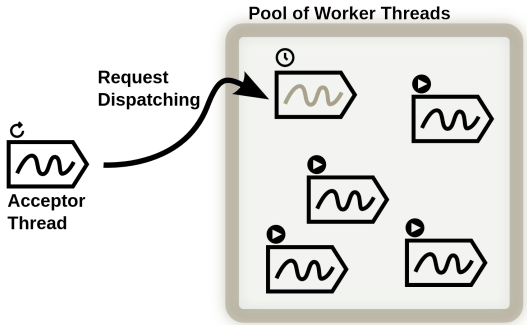


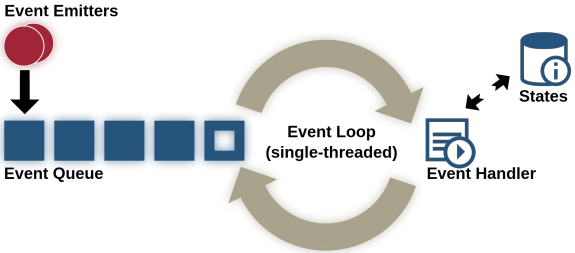


https://berb.github.io/diploma-thesis/community/042_serverarch.html

Thread-based vs event-based programming

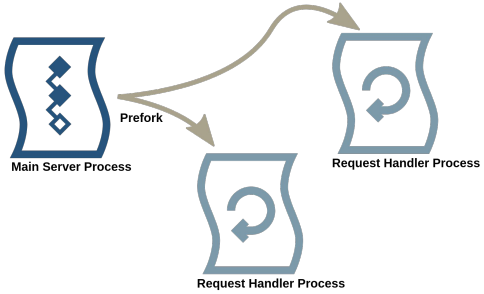


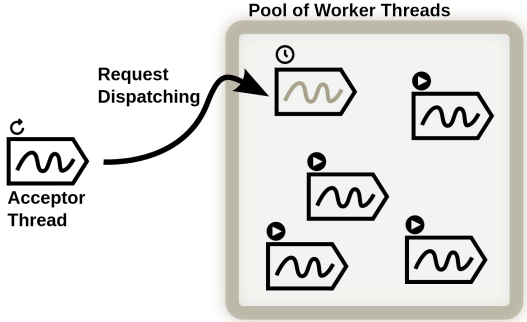


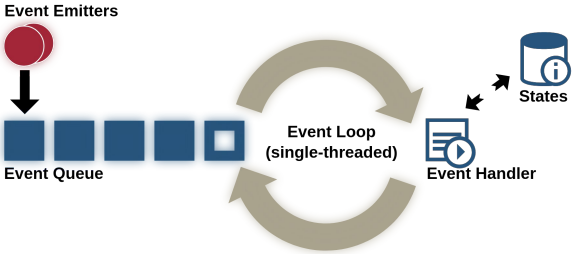


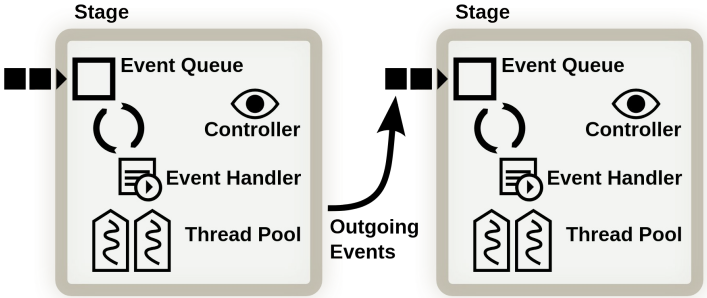
https://berb.github.io/diploma-thesis/community/042_serverarch.html

Thread-based vs event-based programming









https://berb.github.io/diploma-thesis/community/042_serverarch.html

Summary

- What are threads?
- Why are threads useful?
- How does the operating system manage threads?
- How to write a multi-threaded program?
- Alternatives to threads?